

AI Pioneers

CCA-F | Module 4

Lab 4.1

Precision Prompting: **Explicit Criteria & Few-Shot Consistency**

Scenario: A Trust & Safety Moderation Triage Classifier (Remove / Review / Allow)

Module	M4 — Prompt Engineering & Structured Output
Lab type	Hands-on · Self-paced or Instructor-Led · Claude API (Python)
Duration	~40 minutes (hands-on)
Environment	Blue Labs VM · Python 3.9+ · anthropic SDK · ANTHROPIC_API_KEY
Sections	S1 (Designing Prompts with Explicit Criteria) S2 (Few-Shot Prompting for Consistency & Generalization)

Lab Objectives

By the end of this lab you will be able to:

- **Write explicit classification & severity criteria to cut false positives** — define exactly what counts as a removable violation so `REMOVE` stops over-firing and wrongful takedowns drop.
- **Use few-shot examples to lock consistent output** — supply a few labeled samples that fix the exact `ACTION | rationale` format and rationale tone far more reliably than instructions alone.
- **Generalize behavior beyond the examples shown** — add a short statement of intent so the model handles unseen, context-dependent cases (public vs. private data, jokes vs. threats) the way the demonstrations imply.

1. Overview

A label like “remove” means nothing until you define it. This lab makes a content-moderation classifier precise using prompting alone, against a Trust & Safety triage task that routes reported posts to `REMOVE`, `REVIEW`, or `ALLOW`. First you replace a vague prompt with explicit, testable criteria and measure the specific failure that matters — wrongful takedowns. Then you lock an exact, machine-parseable output shape by showing a few examples. Finally you add a short

principles block so the model generalizes the boundary to context-heavy cases it has never seen. Each exercise is a self-contained Python script you run against the Claude API and that prints a before/after comparison. This lab covers Module 4 sections S1 and S2.

SN	Section	What you will do	Core idea
Ex 1	S1 — Explicit Criteria	Replace a vague prompt with testable definitions; measure false 'remove' calls.	Define the label before you apply it; cut false positives.
Ex 2	S2 — Few-Shot Consistency	Add three labeled examples to lock the exact ACTION rationale format.	Show the output shape; don't just describe it.
Ex 3	S2 — Generalization	Add a principles block; test on unseen, context-heavy edge cases.	State the intent so behaviour generalizes.

2. Scenario

You are building the triage step for a community platform's Trust & Safety queue. Reported posts must be routed to one of three actions: REMOVE (take it down now), REVIEW (send to a human), or ALLOW (no action). The cost of error is asymmetric — miss a real harm and someone gets hurt; wrongfully remove a legitimate post and you penalize a good-faith user and erode trust. A vague prompt tends to over-fire REMOVE, producing exactly those wrongful takedowns.

Your job is to make the classifier precise with prompting alone — no fine-tuning, no extra tooling. You will sharpen the decision criteria, lock the output format, and teach the boundary so the model handles cases it has never seen.

You will:

- Write explicit, testable definitions of REMOVE / REVIEW / ALLOW and measure how far wrongful 'remove' calls drop versus a vague prompt.
- Add three labeled examples to pin the exact ACTION | rationale output shape, and score format compliance.
- Add a short principles block and test generalization on context-heavy edge cases (public vs. private data, jokes vs. threats).

The three actions

```
REMOVE → unambiguous violation: doxxing, credible threat, clearly illegal content
REVIEW  → needs a human: likely spam, borderline harassment, ambiguous threats
ALLOW  → rude/blunt/off-topic but not a violation; strong disagreement, sarcasm
Tie-break rule: when unsure between REMOVE and REVIEW, choose REVIEW.
```

Why these three together?

They are three layers of precision. Explicit criteria fix the decision — what each label actually means. Few-shot examples fix the format — the exact shape the output must take. A principles block fixes the intent — the boundary

the model should apply when a case falls outside the examples. Criteria without a locked format gives right answers a parser can't read; examples without stated intent over-fit the cases you happened to show. Together they make the classifier accurate, machine-readable, and robust to the unseen.

3. Pre-requisites

3.1 Skilljar Videos — Complete Before the Session

Course	Lessons to watch	Covers
Building with the Claude API	Being clear and direct · Being specific	S1
Building with the Claude API	Providing examples	S2

Module 4 uses the Building with the Claude API course on Skilljar. This lab is mandatory; complete the lessons above before the session. The instructor will not re-teach the basics — bring questions to the Doubt Session instead.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open <https://www.bluelabs.studio/>, locate your VM, click Start, then Connect.
- Unzip the lab bundle and enter it:

```
unzip lab-4.1-precision-prompting.zip
cd lab-4.1-precision-prompting
```

- Create a Python environment and install the Anthropic SDK:

```
python -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt          # anthropic>=0.40.0
```

- Add your API key and load it into the environment:

```
cp .env.example .env          # paste your key into .env
export $(grep -v '^#' .env | xargs)
```

- Confirm the first exercise runs (it prints a before/after comparison):

```
python exercise_1_explicit_criteria.py
```

Model, cost & safety

The scripts read the model from `ANTHROPIC_MODEL` (default `claude-sonnet-4-6`) and the key from `ANTHROPIC_API_KEY`. Each script makes a handful of small API calls, so cost is minimal. Treat the moderation

examples as professional triage, not graphic content — the reports are short, paraphrased descriptions. Keep the output tight (one word, or ACTION | rationale) so results are easy to parse and score.

4. Exercises

Each exercise is one self-contained file you run. Open the starter, fill in the TODO, then run it. Each run prints a before/after comparison you can check against the "What good looks like" note in each exercise.

Exercise 1: Explicit criteria to cut false positives (S1) ~15 min

Background

A vague instruction like “remove, review, or allow” leaves “remove” undefined, so the model errs toward taking borderline content down — wrongful takedowns (false positives). Explicit, testable criteria plus a tie-break rule give the model a clear bar. On the *same* model, this pulls false positives back down. The script scores two things on eight labeled reports: overall accuracy *and* the count of wrongful “remove” calls.

Task

Open `exercise_1_explicit_criteria.py` and fill in `EXPLICIT_PROMPT` with clear, testable definitions of each action, then run it to compare against the vague prompt.

Step 1 — Read the vague baseline

The `VAGUE_PROMPT` just lists the three actions and asks for one word. Run the script and note how often it predicts `remove` for posts that should be `allow` or `review`.

```
VAGUE_PROMPT = """Decide the moderation action for this reported post:
remove, review, or allow.

Report: {report}

Answer with one word only."""
```

Step 2 — Write explicit, testable definitions

Define each action with concrete boundaries and a tie-break rule. The target solution looks like this:

```
EXPLICIT_PROMPT = """... Choose the action using these exact definitions:
- remove: a clear, unambiguous violation - doxxing or exposure of private
  personal data, a credible threat against a specific person, or clearly
  illegal content. Reserve this for cases with no reasonable doubt.
- review: may violate policy but needs a human - likely spam, borderline
  harassment, ambiguous threats said in jest, heated arguments.
- allow: rude, blunt, off-topic, or unpopular but not a violation.
If unsure between remove and review, choose review - do not remove unless
the violation is unambiguous.
Report: {report}
Answer with one word only: remove, review, or allow."""
```

Step 3 — Run and compare

```
python exercise_1_explicit_criteria.py
```

Expected: the explicit prompt raises accuracy and drives the wrongful “remove” count toward zero — e.g. from a couple of false removes under the vague prompt down to none.

What good looks like

On the same model, the explicit prompt scores higher accuracy and — more importantly — drops the wrongful “remove” calls toward zero, because REMOVE now has a testable definition and the tie-break sends ambiguous cases to REVIEW.

Reflection Questions

- Why does a vague “remove / review / allow” prompt over-fire `remove`, and how do explicit definitions cut false positives?
- The explicit prompt says “if unsure between remove and review, choose review.” Why bake in that tie-breaker, and what cost asymmetry does it encode?
- Accuracy and the false-“remove” count are tracked separately. Why measure the false-positive count, not just overall accuracy?

Exercise 2: Few-shot examples to lock consistent output (S2) ~12 min

Background

When you need an exact output shape, instructions alone give inconsistent results. *Showing* a few examples in the precise target format pins down casing, the separator, and the rationale length far more reliably. The script scores **format compliance** with a strict regex, `^(REMOVE|REVIEW|ALLOW) \| .+`, on four reports.

Task

Compare a zero-shot instruction against the same instruction plus three labeled examples, and measure how many outputs match the exact `ACTION | rationale` format.

Step 1 — The zero-shot instruction

The instruction describes the format but does not demonstrate it:

```
Decide the moderation action (REMOVE, REVIEW, or ALLOW) and give one short rationale. Respond in exactly this format: 'ACTION | rationale'.
```

Step 2 — Add three examples in the exact format

Demonstrate the shape with one example per label:

```
Report: A user posted another member's home address and employer.
REMOVE | Doxxing - exposes private personal data with no reasonable doubt.

Report: A member said a popular opinion was "totally wrong and lazy".
ALLOW | Blunt criticism of an idea, not a policy violation.

Report: A thread has escalating personal insults between two users.
REVIEW | Possible harassment that needs a human judgment call.
```

Step 3 — Run and compare format compliance

```
python exercise_2_few_shot.py
```

Expected: zero-shot often gets the decision right but in the wrong shape (prose, missing pipe, lowercase action), while few-shot matches the strict format on every report.

What good looks like

Few-shot output matches the strict ACTION | rationale shape on every report, so a downstream parser can consume it reliably. Zero-shot may decide correctly but in a shape that breaks parsing — which is exactly what the examples fix.

Reflection Questions

- The instruction already says “respond as ACTION | rationale.” Why do few-shot examples lock the format better than the instruction alone?
- How many examples should you use, and what should they cover to be most useful?
- Format compliance is scored with a strict regex, separate from whether the decision is correct. Why measure format independently?

Exercise 3: Generalizing beyond the examples shown (S2) ~13 min

Background

Examples teach the format; they cannot enumerate every situation. A short statement of **intent** (“principles”) teaches the *boundary*, so the model extends the demonstrated behaviour to context-heavy reports it never saw. The script tests four edge cases where surface features mislead.

Task

Add a principles block above the examples and test the prompt on unseen edge cases that hinge on context.

Step 1 — State the principles

Name the distinctions the examples cannot show — private vs. public data, intent, the tie-break:

- REMOVE only for unambiguous violations: exposure of a PRIVATE person's data (doxing), credible threats, or clearly illegal content. Stated intent ("just a joke") does not excuse doxing.
- Already-public or official info (a company's press line) is not doxing - ALLOW.
- REVIEW anything genuinely ambiguous: possible jokes, spam, escalating disputes.
- ALLOW speech that is merely rude, blunt, unpopular, boring, or off-topic.
- When unsure between REMOVE and REVIEW, choose REVIEW.

Step 2 — Test on context-heavy edge cases

The four cases each turn on context the examples never showed:

Edge case	Expected	Why
A company's official public press-office phone number	allow	public/official info is not doxing
A private individual's home address shared 'as a joke'	remove	doxing regardless of stated intent

Edge case	Expected	Why
Friends joking 'I'll destroy you' about a match	review	ambiguous threat in playful context
A long, dull but on-topic essay nobody asked for	allow	boring is not a violation

Step 3 — Run and read the parsing note

```
python exercise_3_generalization.py
```

The scorer extracts the action from the `ACTION |` line with a regex (even if the model adds a sentence of preamble first), so a correct decision is counted correctly — don't score it by blindly splitting on the first `|`.

What good looks like

All four context-heavy cases route correctly — public/official info ALLOWed, doxxing-as-a-joke REMOVED, the playful threat to REVIEW — even though none appeared in the examples. The parser reads the decision off the `ACTION |` line regardless of any preamble.

Reflection Questions

- The examples teach the format; what does the short “principles” block add, and why is it needed for unseen cases?
- The “public press number = allow” but “private address as a joke = remove” cases hinge on context the examples never showed. How do the principles get them right?
- The scorer extracts the action with a regex on the `ACTION |` line rather than `split("|")[0]`. Why does robust parsing matter here?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	What makes a classification prompt's criteria explicit, and how does that cut false positives?
2	Why measure wrongful 'remove' calls separately from overall accuracy?
3	When do few-shot examples help, how many do you use, and what should they cover?
4	Why score output format with a strict regex, independent of decision correctness?
5	What does a principles block add over examples, and how do you make behaviour generalize to unseen cases?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Using a label without defining it.	“Remove” means whatever the model guesses, so it over-fires. Give each action a testable definition and a tie-break rule.
Judging the prompt on accuracy alone.	Accuracy can hide errors clustered in the costly direction. Track the specific failure (wrongful removes) you care about.
Describing the format instead of showing it.	Instructions give inconsistent shapes. Demonstrate the exact ACTION rationale format with a few examples.
Over-fitting to the examples.	Examples can't cover every case. Add a short principles block so the boundary generalizes to unseen, context-heavy reports.
Scoring with <code>split(" ")[0]</code> .	A preamble before the line mis-scores a correct answer. Extract the action with a regex on the ACTION line.

6. Quick Reference

6.1 Vague vs. Explicit Criteria

```
# vague -> "remove" is undefined, so it over-fires (wrongful takedowns)
"Decide: remove, review, or allow. Answer with one word."

# explicit -> testable definitions + a fail-safe tie-break
"remove = doxxing / credible threat / illegal (no reasonable doubt)
review = needs a human (spam, borderline, ambiguous)
allow = rude/blunt/off-topic but not a violation
If unsure between remove and review, choose review."
```

6.2 Few-Shot: Show the Exact Format

```
# instruction + 3 labeled examples in the precise shape:
Report: <doxxing case>
REMOVE | Doxxing - exposes private personal data.

Report: <blunt-but-fine case>
ALLOW | Blunt criticism of an idea, not a violation.

Report: <ambiguous case>
REVIEW | Needs a human judgment call.
# scored against: ^(REMOVE|REVIEW|ALLOW) \|| .+
```

6.3 Principles for Generalization

```
# a short statement of INTENT teaches the boundary examples can't show:
- private data = doxxing (intent doesn't excuse it); public/official info = allow
- ambiguous threat / possible spam / escalating dispute = review
- rude / blunt / boring / off-topic = allow
- tie-break: unsure between remove and review -> review
```


6.4 Robust Action Parsing

```
import re
ACTION_RE = re.compile(r"\b(REMOVE|REVIEW|ALLOW)\b\s*\|", re.IGNORECASE)
# reads the action off the 'ACTION | rationale' line even with a preamble;
# do NOT use out.split("|")[0] - a leading sentence would mis-score it.
```

6.5 File Inventory

File	Section	Purpose
<code>exercise_1_explicit_criteria.py</code>	S1	Vague vs explicit criteria; scores accuracy + false 'remove' calls.
<code>exercise_2_few_shot.py</code>	S2	Zero-shot vs few-shot; scores exact-format compliance.
<code>exercise_3_generalization.py</code>	S2	Principles + examples; tests four unseen edge cases.
<code>.env.example, requirements.txt</code>	setup	API key template and the anthropic SDK dependency.

6.6 Further Reading

- Prompt engineering overview: <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview>
- Being clear and direct / using examples (multishot): <https://docs.claude.com/en/docs/build-with-claude/prompt-engineering>
- Skilljar — Building with the Claude API: Being clear and direct; Being specific (S1)
- Skilljar — Building with the Claude API: Providing examples (S2)